

Learned Page Replacement Under a Workload Shift

CSE-307 Operating Systems, Part B Term Paper (Track 1)

Sayed Md Shafiqul Islam

ID: 202414011 Section A CSE 24

4 October 2026

1 Problem Framing

Page replacement decides which resident page to evict when a reference faults and memory is full. FIFO and LRU are fixed rules: FIFO ignores usage, and LRU assumes that the recent past predicts the near future [3]. Belady's optimal policy (OPT) evicts the page whose next use is farthest away. It needs future knowledge, so it is only a lower bound on faults that online policies are measured against [1, 2]. Real workloads change phase, so a policy that suits one phase can suit the next one badly.

This paper asks two questions. (1) Can a small learned model that predicts page reuse from simple features beat FIFO and LRU? (2) What happens to each policy, including the learned one, when the access pattern shifts half-way through a run? The learned component is deliberately lightweight (a depth-4 decision tree), in the spirit of ML-driven caching work such as LeCaR [4].

2 Implementation

All code is Python 3 with NumPy [6] and scikit-learn [5]. A single simulator replays a page-reference string against F frames, and each policy only supplies a victim-selection rule.

- **FIFO**: evicts the page loaded earliest. **LRU**: evicts the least recently referenced page. **OPT**: evicts the resident page with the farthest next reference, computed from a precomputed next-use array.
- **Correctness checks**: on the textbook reference string with 3 frames [3] the simulator reproduces 15 (FIFO), 12 (LRU) and 9 (OPT) faults, and a test confirms that OPT never has more faults than any other policy on the synthetic traces.
- **Learned policy**. At every eviction, each resident page is described by four features an OS could keep as page metadata: *recency* (accesses since last use), *frequency* (uses in the last 256 accesses), *age in memory*, and *last reuse gap*. A decision tree (max depth 4, min leaf 20) predicts whether the page will be referenced again within $H=64$ accesses. The page with the lowest predicted reuse probability is evicted, with ties broken by recency.
- **Two variants**. *Learned-static* is trained offline on three pre-shift traces (different seeds from all test traces) and then frozen. *Learned-adaptive* starts from the same model and retrain every 1000 accesses on the most recent 6000 labelled decisions. A decision is labelled only H accesses after it was made, so no future information leaks into training.

3 Experimental Setup

Each trace has 20,000 references over $V=256$ virtual pages, with a deliberate shift at reference 10,000. **Phase 1 (locality-heavy/sequential)**: 70% of references go to a 20-page hot set with Zipf weights, and 30% are a cyclic sequential scan over the other 236 pages. The scan pollutes recency-based policies. **Phase 2 (random/bursty)**: uniform-random references interleaved with bursts of 10–40 references to a random 8-page region (about 40% of references fall inside bursts). The main configuration uses $F=32$ frames and 10 random seeds; memory sizes 16, 32 and 64 are also tested. Hits and faults are counted separately for each half. Results are mean $\pm$ standard deviation over seeds. Everything is reproducible with `python run_experiments.py`.

4 Results

Before the shift the learned policies clearly win among online policies: 67.3% (static) and 69.2% (adaptive) against 60.5% for LRU and 52.7% for FIFO, within 3.8 and 1.9 pp of OPT's 71.1%. After the shift the picture reverses. FIFO and LRU

Table 1: Hit ratio and faults per 10,000 references before/after the shift ($F=32$, 10 traces). “Drop” is the loss in hit ratio in percentage points (pp); “Gap” is the distance to OPT after the shift.

Policy	Hit% before	Hit% after	Drop (pp)	Faults before	Faults after	Gap (pp)
FIFO	52.7 ± 0.5	38.3 ± 1.1	14.4	4732	6171	20.8
LRU	60.5 ± 0.5	38.4 ± 1.2	22.1	3946	6158	20.7
OPT	71.1 ± 0.3	59.1 ± 0.7	12.0	2890	4086	0.0
Learned-static	67.3 ± 0.4	34.2 ± 1.2	33.1	3267	6578	24.9
Learned-adaptive	69.2 ± 0.5	36.0 ± 1.0	33.2	3080	6396	23.1

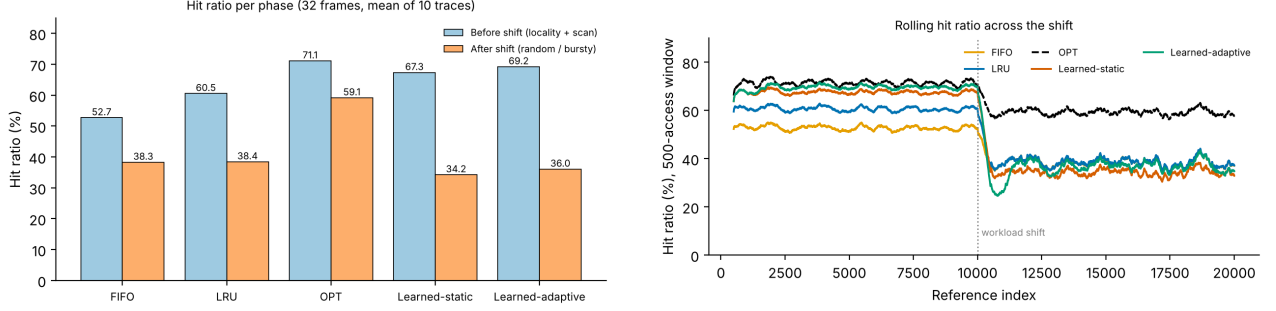


Figure 1: Left: hit ratio per phase. Right: 500-access rolling hit ratio averaged over 10 traces; the dotted line marks the shift.

both fall to about 38.4%, while Learned-static falls to 34.2% and Learned-adaptive to 36.0%, so both learned variants end *below* LRU. The learned policies suffer the largest drop (about 33 pp), against 22.1 pp for LRU, 14.4 pp for FIFO and 12.0 pp for OPT. Figure 1 (right) shows the adaptive variant falling to roughly 25% right after the shift before recovering to the level of LRU within about 2000 references.

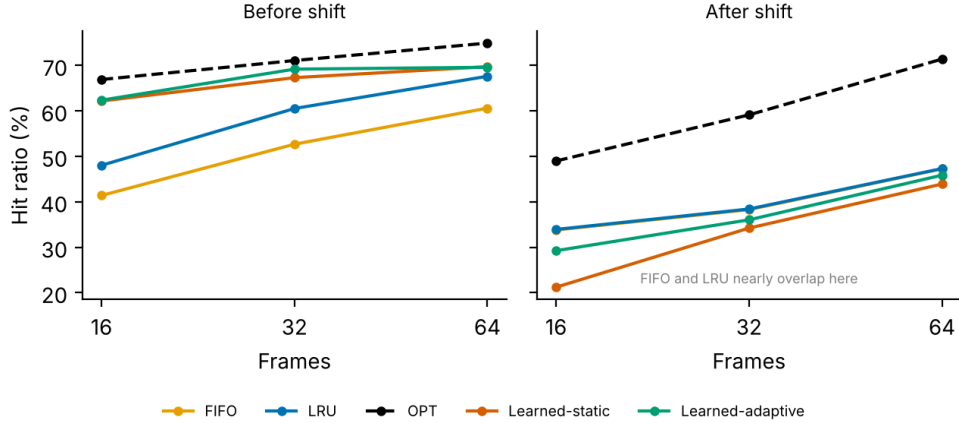


Figure 2: Hit ratio vs. number of frames, before and after the shift.

Memory size matters (Figure 2). With 16 frames the frozen model is badly hurt after the shift (21.2% against 33.9% for LRU), whereas with 64 frames the gap narrows (43.9% against 47.3%). OPT stays far ahead after the shift at every size, so a large part of the available benefit is future knowledge that no online policy has.

5 Analysis: What Changed Under the Shift

Why the learned policy wins first. The trained tree is almost entirely a frequency rule (99.4% of feature importance): pages used at least twice in the recent window are kept, and pages seen once are evicted first. In phase 1 this is exactly right. Scan pages are touched once per cycle and are useless, but they flood an LRU list and push out warm hot pages. Frequency recognises scan pages and protects the hot set, which is why the learned policy beats LRU there.

Why it degrades the most. The model learned a property of the old workload (“high frequency means reuse soon”), and that property stops being true. After the shift, most pages are random and have frequency near one, so decisions fall back to recency, which is LRU-like. Pages from finished bursts, however, keep a high frequency count and are still protected. We measured this directly (10 traces, post-shift): on average 12.2 of the 32 frames hold pages the model rates as likely to be reused although they have not been touched for over 100 accesses, and the frozen model evicts a page that is needed

again within 16 accesses in 10.8% of evictions, against 4.9% for LRU. It is plausible that the first accesses of a new burst, which still have frequency one, are evicted too early, but we did not isolate this effect. Stale frequency information of this kind is a known weakness of frequency-based policies.

Why FIFO and OPT “drop” less. FIFO’s smaller drop (14.4 pp) is mostly a floor effect: it was already the weakest policy before the shift, and after the shift it is as good as LRU. OPT’s 12.0 pp drop shows that part of the loss is unavoidable, because random references offer less reusable locality than the hot set did. The remaining gap to OPT is the cost of not knowing the future, and it is about 21 pp for FIFO/LRU and 23–25 pp for the learned variants.

Does adapting help? Partly. Retraining recovers 1.8 pp over the frozen model after the shift (36.0% vs. 34.2%) and 8.7 pp over LRU before it, but it does not beat LRU after the shift. It learns from labels that arrive H references late, and its training window still contains pre-shift decisions, hence the dip in Figure 1. Retraining every 500, 1000 and 2000 references gives post-shift hit ratios of 36.5%, 36.0% and 33.6%, so faster adaptation helps but only modestly.

6 Limitations and Conclusion

The traces are synthetic, with a single shift and hand-chosen parameters, so the absolute numbers will differ on real traces and the ranking after the shift depends on how bursty phase 2 is. The model is a small tree evaluated in a simulator, and its run-time cost and metadata overhead inside a kernel are not modelled. OPT is an offline reference only.

In summary, a tiny learned eviction rule beat LRU by 7–9 pp while the workload matched its training data, but it also lost the most when the pattern changed, finishing 2–4 pp *below* LRU after the shift. Online retraining reduces the damage without removing it. The practical lesson is that a learned OS heuristic needs drift detection and a safe fallback (such as LRU), not just a good offline accuracy.

References

- [1] L. A. Belady, “A study of replacement algorithms for a virtual-storage computer,” *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966.
- [2] R. L. Mattson, J. Gecsei, D. R. Slutz, and I. L. Traiger, “Evaluation techniques for storage hierarchies,” *IBM Systems Journal*, vol. 9, no. 2, pp. 78–117, 1970.
- [3] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Wiley, 2018.
- [4] G. Vietri *et al.*, “Driving cache replacement with ML-based LeCaR,” in *Proc. USENIX HotStorage*, 2018.
- [5] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [6] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.